GeoSpatialIndexBenchmark

buildGeoSpatialIndex

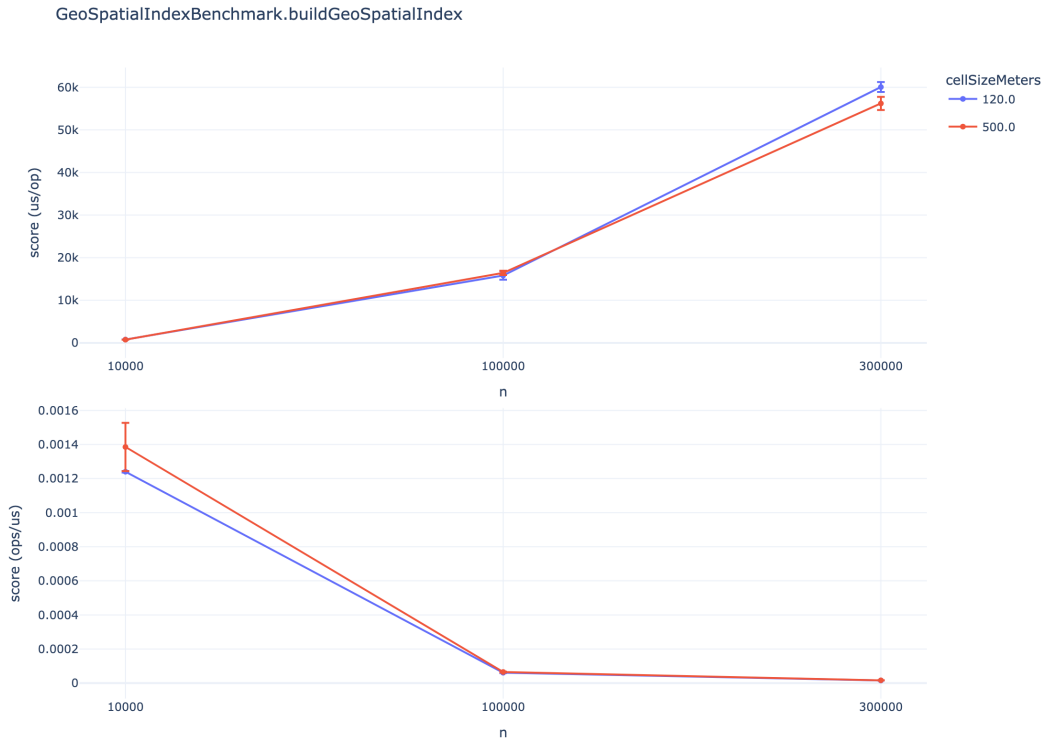


Figure 1: score

JMH Results avgt

n cellSizeMeters	120.0	500.0
10000	788.476948 +- 11.836680 us/op	743.172499 +- 2.673992 us/op
100000	15805.132207 +- 983.466161 us/op	16431.167851 +- 511.716338 us/op
300000	60057.805939 +- 1153.522394 us/op	56212.428657 +- 1543.791264 us/op

thrpt

n cellSizeMeters	120.0	500.0
10000	0.001240 +- 0.000005 ops/us	0.001386 +- 0.000142 ops/us
100000	0.000061 +- 0.000002 ops/us	0.000066 +- 0.000002 ops/us
300000	0.000016 +- 0.000000 ops/us	0.000017 +- 0.000000 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.048	10.052
100000	17.141	17.082
300000	17.509	17.238

cpu_machine_total_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.609	10.555
100000	18.229	18.238
300000	18.447	18.342

cpu_samples_per_sec

n cellSizeMeters	120.0	500.0
10000	101.792	101.708
100000	173.771	172.104
300000	178.271	176.396

alloc_events_per_sec

n cellSizeMeters	120.0	500.0
10000	3677.979	3380.979
100000	1666.250	1783.750
300000	1342.354	1224.380

alloc_events_total

n cellSizeMeters	120.0	500.0
10000	88272	81144
100000	39990	42810
300000	32216	30610

alloc_tlab_share_pct

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes

- Построение spatial-индекса заметно дороже naive-версии, потому что в GeoSpatialIndex.put(…) происходит запись в byId и вставка в QuadTree. Если объект уже был, то ещё и удаление старого из дерева. Старый объект извлекается из byId, при наличии удаляется из дерева, затем новый объект вставляется обратно. У naive-версии никакой второй структуры нет, поэтому построение почти сводится к работе одного HashMap. Отсюда и большая разница.
- Что можно улучшить в коде: задать начальную ёмкость HashMap под ожидаемый размер, чтобы уменьшить rehash, сделать bulk-load путь вместо чисто поэлементного put.

buildNaiveIndex

JMH Results avgt

n cellSizeMeters	120.0	500.0
10000	130.157776 +- 0.530767 us/op	129.880918 +- 0.386195 us/op
100000	4003.269476 +- 181.996268 us/op	4140.655466 +- 260.561227 us/op
300000	13125.459990 +- 596.855501 us/op	13164.878176 +- 405.253234 us/op

thrpt

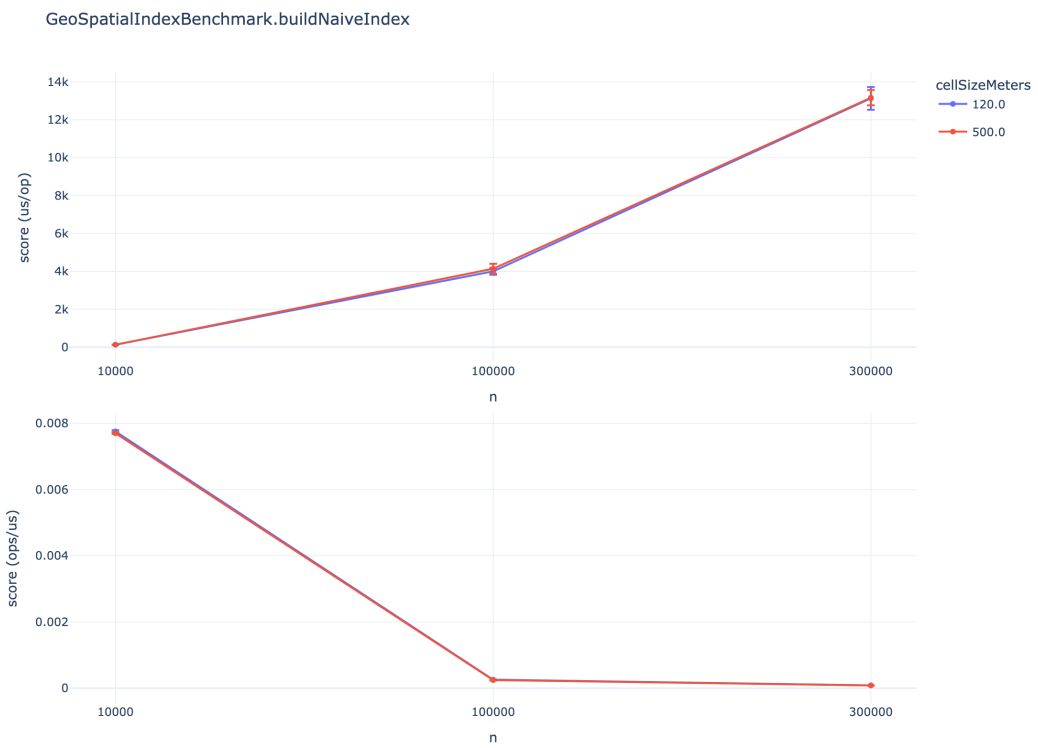


Figure 2: score

n cellSizeMeters	120.0	500.0
10000	0.007756 +- 0.000036 ops/us	0.007701 +- 0.000026 ops/us
100000	0.000257 +- 0.000010 ops/us	0.000245 +- 0.000014 ops/us
300000	0.000083 +- 0.000002 ops/us	0.000083 +- 0.000002 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.160	10.190
100000	34.302	33.373
300000	35.703	36.089

cpu_machine_total_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.869	11.032
100000	35.656	34.888
300000	37.904	38.045

cpu_samples_per_sec

n cellSizeMeters	120.0	500.0
10000	103.250	104.000
100000	351.146	341.917
300000	359.979	358.167

alloc_events_per_sec

n cellSizeMeters	120.0	500.0
10000	14179.792	13964.104
100000	4380.521	4680.062
300000	3979.458	4115.250

alloc_events_total

n cellSizeMeters	120.0	500.0
10000	340315	335138
100000	105132	112322
300000	95507	98766

alloc_tlab_share_pct

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes Этот метод лучший по стоимости построения. По сути он в несколько раз быстрее spatial-версии на всех размерах входа, потому что он не поддерживает пространственную структуру. Он просто кладёт объекты в map.

nearbyQueryGeoSpatialIndex

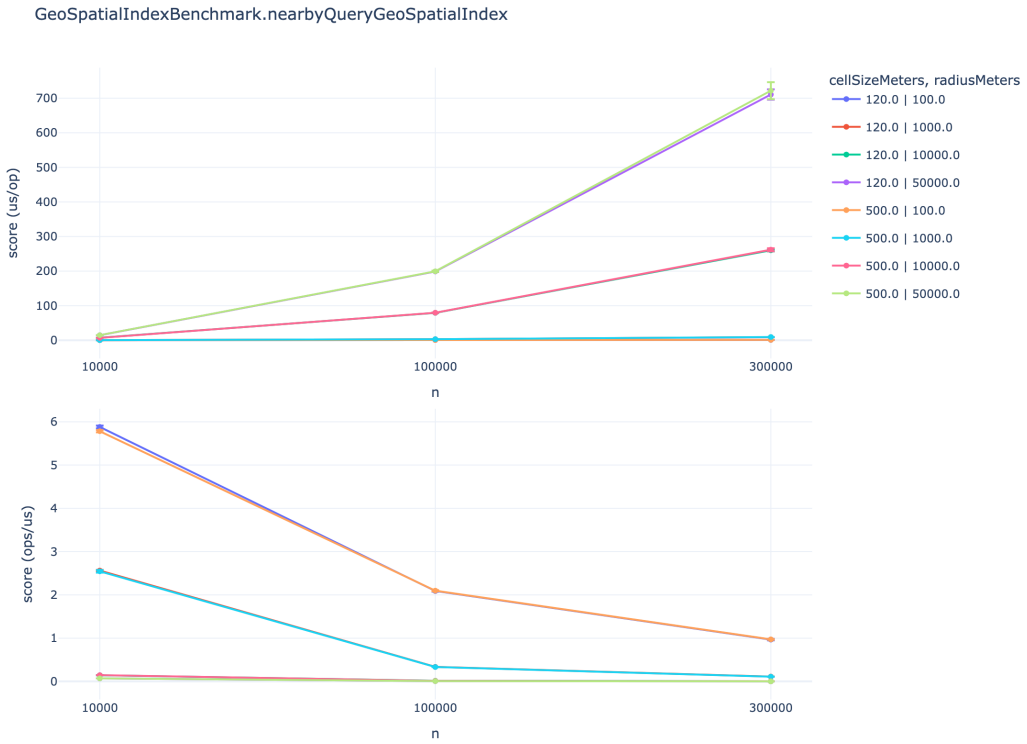


Figure 3: score

JMH Results avgt

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	0.169884 +- 0.001592 us/op	0.173269 +- 0.001193 us/op
100000	0.478847 +- 0.002499 us/op	0.479022 +- 0.002271 us/op
300000	1.035520 +- 0.005491 us/op	1.014204 +- 0.010591 us/op

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	0.389206 +- 0.001212 us/op	0.390170 +- 0.001066 us/op
100000	3.008754 +- 0.008532 us/op	3.003476 +- 0.014504 us/op
300000	9.066168 +- 0.051865 us/op	9.141031 +- 0.052797 us/op

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	7.071667 +- 0.027510 us/op	6.951704 +- 0.021852 us/op
100000	79.249304 +- 0.516472 us/op	79.645248 +- 0.372761 us/op
300000	260.041727 +- 3.082972 us/op	261.825920 +- 4.343436 us/op

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	14.595304 +- 0.031988 us/op	14.648007 +- 0.100822 us/op
100000	198.815033 +- 1.757781 us/op	199.446502 +- 1.889692 us/op
300000	710.726247 +- 14.849346 us/op	721.727633 +- 24.500483 us/op

thrpt

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	5.881574 +- 0.031021 ops/us	5.782459 +- 0.025813 ops/us
100000	2.089066 +- 0.014263 ops/us	2.095947 +- 0.011960 ops/us
300000	0.964744 +- 0.016092 ops/us	0.970350 +- 0.007547 ops/us

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	2.562661 +- 0.011977 ops/us	2.543486 +- 0.028499 ops/us
100000	0.334722 +- 0.000931 ops/us	0.333300 +- 0.001131 ops/us
300000	0.110431 +- 0.000490 ops/us	0.108867 +- 0.000425 ops/us

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	0.142413 +- 0.000937 ops/us	0.144880 +- 0.000367 ops/us
100000	0.012578 +- 0.000096 ops/us	0.012534 +- 0.000063 ops/us
300000	0.003814 +- 0.000046 ops/us	0.003756 +- 0.000068 ops/us

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	0.068560 +- 0.000212 ops/us	0.069230 +- 0.000105 ops/us
100000	0.005027 +- 0.000077 ops/us	0.005093 +- 0.000032 ops/us
300000	0.001405 +- 0.000029 ops/us	0.001388 +- 0.000045 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	9.998	10.000
100000	9.998	9.996
300000	10.002	10.007

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	10.002	9.996
100000	10.000	9.998
300000	10.002	10.004

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	9.998	10.002
100000	10.004	10.009
300000	10.024	10.020

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	10.000	10.002
100000	10.004	10.017
300000	10.028	10.022

cpu_machine_total_avg_pct

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	10.549	10.490
100000	10.418	10.489
300000	10.587	10.525

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	10.474	10.430
100000	10.464	10.501
300000	10.481	10.469

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	10.511	11.049
100000	10.511	10.407
300000	10.541	10.464

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	10.517	10.459
100000	13.263	10.502
300000	10.548	10.464

cpu_samples_per_sec

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	101.521	101.833
100000	101.042	102.000
300000	101.021	101.562

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	101.729	102.083
100000	101.583	101.771
300000	101.521	102.167

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	101.833	102.042
100000	101.438	102.062
300000	101.646	102.250

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	102.188	102.562
100000	101.854	102.062
300000	101.375	102.250

alloc_events_per_sec

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	2488.125	2195.500
100000	1269.125	1272.604
300000	1074.271	1068.396

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	2074.750	2047.562
100000	1446.938	1459.979
300000	1340.708	1330.521

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	1977.625	1977.729
100000	1639.250	1637.000
300000	1517.271	1503.771

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	2209.458	2223.854
100000	1468.771	1446.062
300000	1274.146	1275.708

alloc_events_total

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	59715	52692
100000	30459	30542
300000	25782	25642

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	49794	49142
100000	34726	35040
300000	32177	31932

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	47463	47466
100000	39342	39288
300000	36414	36090

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	53027	53372
100000	35250	34706
300000	30580	30617

alloc_tlab_share_pct

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes

- По коду findNearby(…) считает ограничивающий прямоугольник по широте и долготе, строит одно или два окна по долготе, просит дерево собрать только кандидатов из пересекающихся диапазонов, затем для этих кандидатов считает точную дистанцию и сортирует итоговые результаты. Именно поэтому nearby-query не зависит от n так же жёстко, как в naive-подходе. Алгоритм не смотрит на все точки, а сначала резко сужает область поиска. Spatial-index особенно хорош, когда запросы локальные и это полностью соответствует его назначению.
- У nearbyQueryGeoSpatialIndex аллокационная активность очень заметная. Это объясняется кодом: создаётся List<GeoObject> candidates, потом List<GeoSearchResult> results, потом HashSet seenIds, плюс создаются объекты GeoSearchResult.
- Нужно проверить, нужен ли seenIds. Сейчас он добавляет лишнюю память. Если tree.collectInRange(…) и разбиение окон гарантируют отсутствие дублей, HashSet можно убрать. Это даст прямой выигрыш по allocation и CPU.

nearbyQueryNaive

JMH Results avgt

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	429.209935 +- 2.177532 us/op	428.598071 +- 1.389971 us/op
100000	4193.912208 +- 14.156143 us/op	4206.160712 +- 17.618530 us/op
300000	12309.664600 +- 48.594508 us/op	12384.181094 +- 44.779375 us/op

`radiusMeters=1000.0`

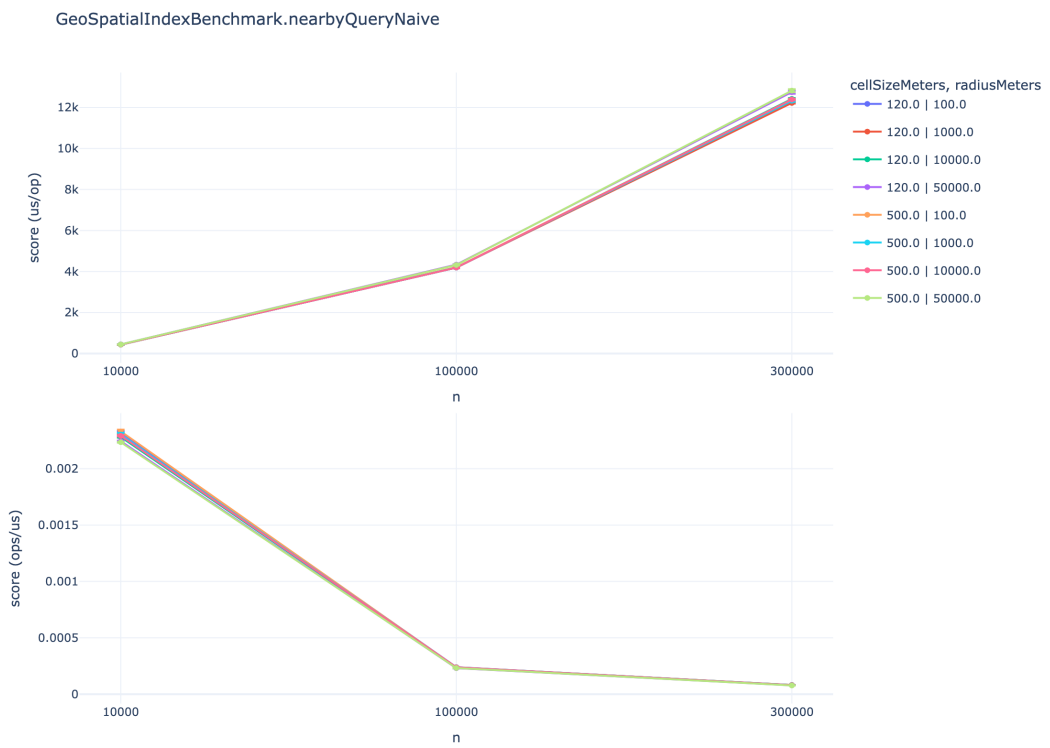


Figure 4: score

n cellSizeMeters	120.0	500.0
10000	434.110734 +- 3.952315 us/op	436.227811 +- 3.726947 us/op
100000	4194.600401 +- 31.334445 us/op	4191.667817 +- 26.600088 us/op
300000	12222.023098 +- 29.837949 us/op	12324.653463 +- 48.199617 us/op

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	439.308097 +- 1.512193 us/op	437.848529 +- 2.666342 us/op
100000	4187.419834 +- 19.115302 us/op	4188.056215 +- 20.492579 us/op
300000	12420.886821 +- 41.865998 us/op	12389.293658 +- 32.697344 us/op

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	446.003476 +- 0.778670 us/op	444.992725 +- 0.689261 us/op
100000	4337.363102 +- 14.329838 us/op	4318.024602 +- 15.689984 us/op
300000	12750.493952 +- 78.637751 us/op	12820.192882 +- 72.276984 us/op

thrpt

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	0.002323 +- 0.000011 ops/us	0.002331 +- 0.000015 ops/us
100000	0.000239 +- 0.000001 ops/us	0.000238 +- 0.000001 ops/us
300000	0.000081 +- 0.000000 ops/us	0.000081 +- 0.000000 ops/us

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	0.002292 +- 0.000016 ops/us	0.002302 +- 0.000016 ops/us
100000	0.000237 +- 0.000002 ops/us	0.000238 +- 0.000002 ops/us
300000	0.000081 +- 0.000000 ops/us	0.000081 +- 0.000000 ops/us

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	0.002282 +- 0.000006 ops/us	0.002291 +- 0.000005 ops/us
100000	0.000237 +- 0.000002 ops/us	0.000239 +- 0.000001 ops/us
300000	0.000080 +- 0.000000 ops/us	0.000081 +- 0.000000 ops/us

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	0.002243 +- 0.000005 ops/us	0.002235 +- 0.000004 ops/us
100000	0.000230 +- 0.000001 ops/us	0.000232 +- 0.000001 ops/us
300000	0.000079 +- 0.000001 ops/us	0.000078 +- 0.000000 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	9.996	9.998
100000	10.004	10.007
300000	10.000	9.996

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	9.994	9.996
100000	10.050	10.054
300000	10.017	10.028

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	9.994	9.989
100000	10.000	9.996
300000	9.994	9.996

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	9.996	9.996
100000	10.002	9.996
300000	9.994	9.998

cpu_machine_total_avg_pct

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	10.514	10.338
100000	10.687	10.376
300000	10.563	10.411

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	10.355	10.390
100000	10.417	10.510
300000	10.376	10.432

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	10.331	10.444
100000	11.142	10.407
300000	10.421	10.476

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	10.457	10.478
100000	10.410	10.503
300000	10.471	11.330

cpu_samples_per_sec

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	101.812	101.938
100000	102.188	102.042
300000	102.646	102.167

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	101.917	101.583
100000	102.250	102.375
300000	102.500	102.312

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	101.750	101.854
100000	101.875	101.938
300000	102.250	102.229

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	101.917	101.854
100000	101.979	102.021
300000	102.604	102.333

alloc_events_per_sec

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	0.250	0.250
100000	0.125	0.021
300000	0.021	0.042

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	0.688	0.708
100000	0.354	0.375
300000	0.333	0.458

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	8.479	8.750
100000	8.292	8.000
300000	7.917	7.688

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	29.208	28.354
100000	28.854	27.583
300000	27.292	31.333

alloc_events_total

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	6	6
100000	3	0
300000	0	1

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	16	17
100000	8	9
300000	8	11

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	204	210
100000	199	192
300000	190	184

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	701	680
100000	692	662
300000	655	752

alloc_tlab_share_pct

`radiusMeters=100.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

`radiusMeters=1000.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

`radiusMeters=10000.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

`radiusMeters=50000.0`

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes Naive-реализация демонстрирует предсказуемое, но плохо масштабируемое поведение: стоимость nearby-query определяется почти исключительно полным перебором коллекции и потому растёт почти линейно с увеличением числа объектов.

removeThenPutGeoSpatialIndex

JMH Results avgt

n cellSizeMeters	120.0	500.0
10000	0.119184 +- 0.001080 us/op	0.118545 +- 0.001131 us/op
100000	0.215727 +- 0.008854 us/op	0.206293 +- 0.006739 us/op

n cellSizeMeters	120.0	500.0
300000	0.369371 +- 0.028320 us/op	0.365688 +- 0.018865 us/op

thrpt

n cellSizeMeters	120.0	500.0
10000	8.430082 +- 0.124735 ops/us	8.516366 +- 0.074304 ops/us
100000	4.788463 +- 0.159167 ops/us	4.723706 +- 0.176103 ops/us
300000	2.824770 +- 0.363832 ops/us	2.772078 +- 0.137996 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.063	10.101
100000	11.186	10.709
300000	13.382	13.666

cpu_machine_total_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.691	10.541
100000	11.554	11.233
300000	13.831	14.201

cpu_samples_per_sec

n cellSizeMeters	120.0	500.0
10000	103.521	103.083
100000	114.438	107.833
300000	134.560	136.520

alloc_events_per_sec

n cellSizeMeters	120.0	500.0
10000	1391.833	1402.875
100000	831.271	833.479
300000	523.900	518.400

alloc_events_total

n cellSizeMeters	120.0	500.0
10000	33404	33669
100000	19950	20004
300000	13098	12960

alloc_tlab_share_pct

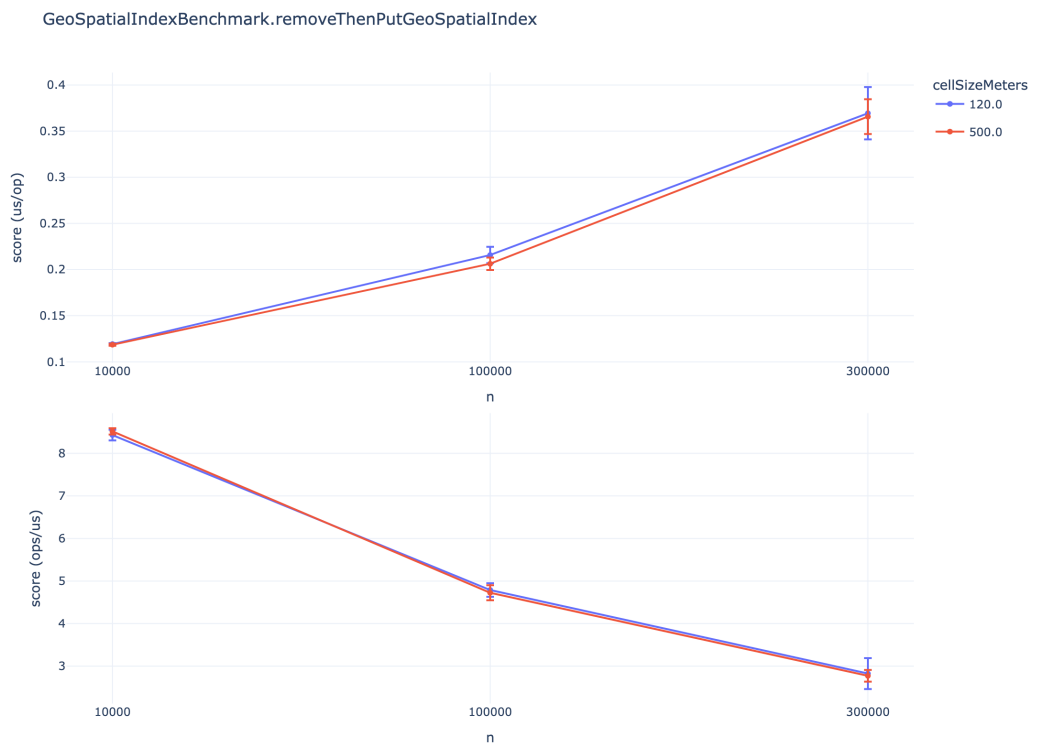


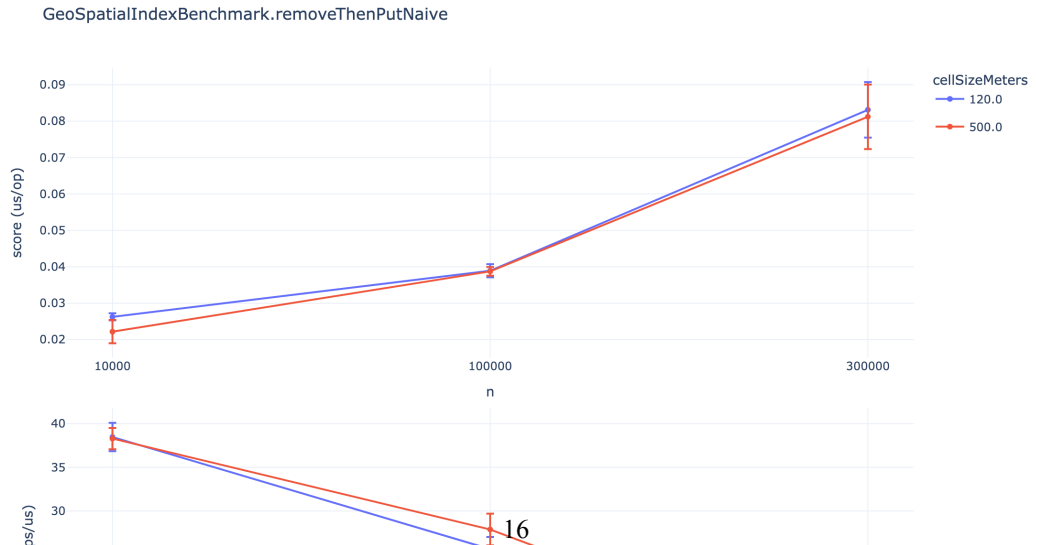
Figure 5: score

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes

- Метод остаётся очень быстрым в абсолютных значениях, но ощутимо проигрывает naive-версии, потому что это не просто “удалить и вставить в map”. Здесь spatial index поддерживает две структуры: map и дерево. Это и даёт дополнительную цену.
- Можно хранить ссылку на leaf node или spatial position вместе с объектом. Тогда remove можно сделать дешевле, без повторного поиска пути от корня. Добавить merge узлов после серии удалений.

removeThenPutNaive



n cellSizeMeters	120.0	500.0
300000	12.266564 +- 2.111176 ops/us	11.671418 +- 0.749200 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.510	10.549
100000	12.189	11.982
300000	18.471	17.205

cpu_machine_total_avg_pct

n cellSizeMeters	120.0	500.0
10000	11.112	11.233
100000	12.727	12.554
300000	19.170	17.790

cpu_samples_per_sec

n cellSizeMeters	120.0	500.0
10000	106.458	107.333
100000	122.396	120.979
300000	178.260	166.080

alloc_events_per_sec

n cellSizeMeters	120.0	500.0
10000	6398.104	6240.104
100000	4477.667	4256.229
300000	1818.220	1952.900

alloc_events_total

n cellSizeMeters	120.0	500.0
10000	153554	149762
100000	107464	102150
300000	45456	48822

alloc_tlab_share_pct

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes

- Naive-структура поддерживает только map. Удаление и вставка в HashMap это намного дешевле, чем аналогичная операция в map плюс quadtree.

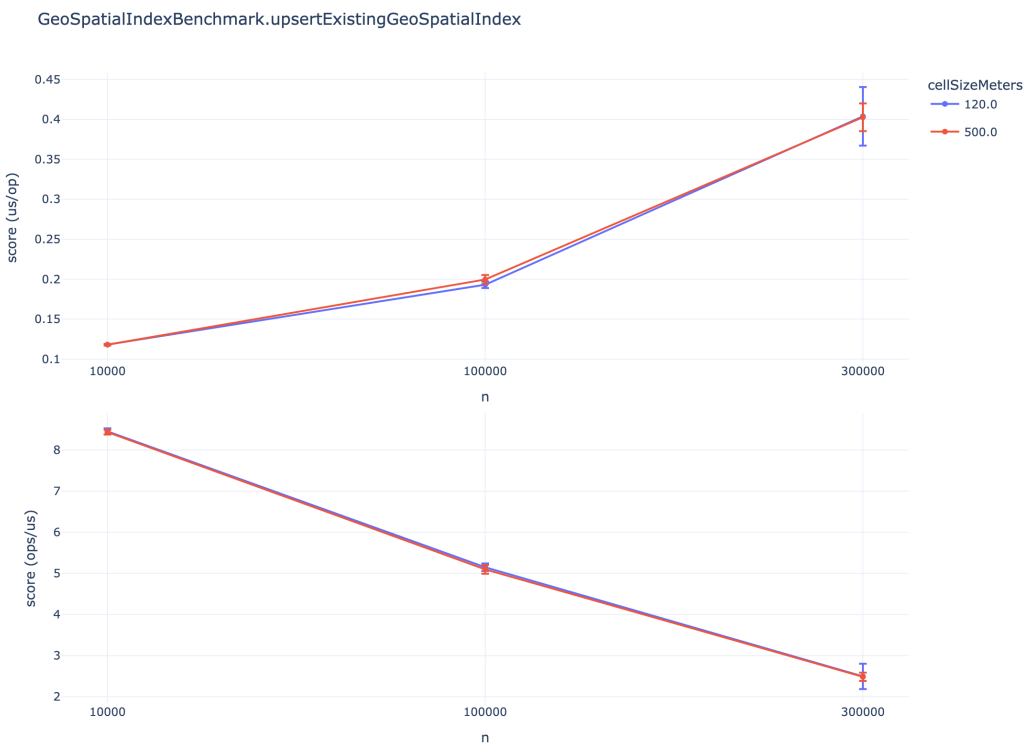


Figure 7: score

upsertExistingGeoSpatialIndex

JMH Results avgt

n cellSizeMeters	120.0	500.0
10000	0.118135 +- 0.000878 us/op	0.118005 +- 0.000827 us/op
100000	0.193077 +- 0.004109 us/op	0.199620 +- 0.005566 us/op
300000	0.403837 +- 0.036569 us/op	0.402623 +- 0.017347 us/op

thrpt

n cellSizeMeters	120.0	500.0
10000	8.454526 +- 0.074002 ops/us	8.436241 +- 0.057624 ops/us
100000	5.150097 +- 0.092101 ops/us	5.097852 +- 0.108403 ops/us
300000	2.494703 +- 0.308221 ops/us	2.482845 +- 0.099825 ops/us

Async Profiler Metrics by Params/Mode cpu_jvm_user_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.063	10.065
100000	10.494	10.576
300000	12.424	11.734

cpu_machine_total_avg_pct

n cellSizeMeters	120.0	500.0
10000	10.452	10.432
100000	10.872	11.030
300000	12.908	12.151

cpu_samples_per_sec

n cellSizeMeters	120.0	500.0
10000	102.625	103.062
100000	107.708	108.229
300000	124.320	117.800

alloc_events_per_sec

n cellSizeMeters	120.0	500.0
10000	913.958	904.125
100000	563.458	584.083
300000	343.640	331.260

alloc_events_total

n cellSizeMeters	120.0	500.0
10000	21935	21699
100000	13523	14018
300000	8591	8282

alloc_tlab_share_pct

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes

- Поведение очень похоже на removeThenPutGeoSpatialIndex. Upsert существующего объекта фактически и есть remove old from tree + insert new into tree. Можно, если координаты не изменились, не трогать дерево вообще. Если координаты изменились, но leaf не изменился, то менять только содержимое.

upsertExistingNaive

JMH Results avgt

n cellSizeMeters	120.0	500.0
10000	0.014739 +- 0.000288 us/op	0.014605 +- 0.000182 us/op
100000	0.030242 +- 0.001587 us/op	0.029868 +- 0.001701 us/op
300000	0.056194 +- 0.005836 us/op	0.058294 +- 0.002815 us/op

thrpt

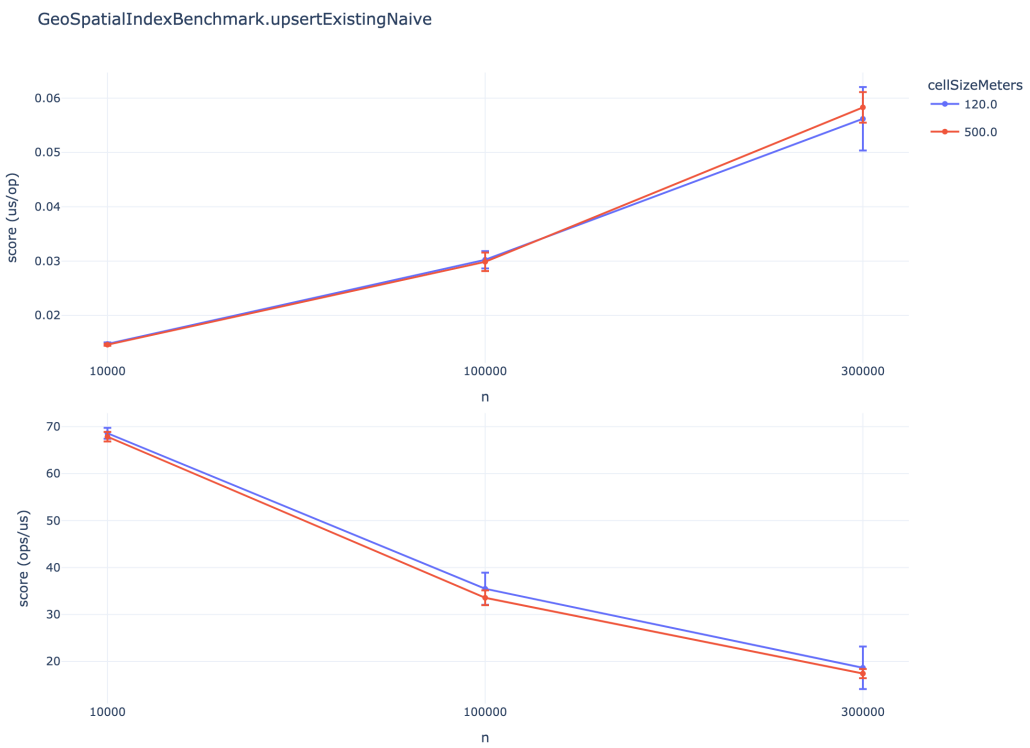


Figure 8: score

n cellSizeMeters	120.0	500.0
10000	68.604328 +- 1.156622 ops/us	67.867834 +- 1.020715 ops/us
100000	35.476320 +- 3.425025 ops/us	33.559664 +- 1.580847 ops/us
300000	18.645135 +- 4.521627 ops/us	17.413719 +- 0.985508 ops/us

Async Profiler Metrics by Params/Mode **cpu_jvm_user_avg_pct**

n cellSizeMeters	120.0	500.0
10000	10.389	10.393
100000	11.116	11.097
300000	11.786	11.851

cpu_machine_total_avg_pct

n cellSizeMeters	120.0	500.0
10000	11.093	10.837
100000	11.692	11.692
300000	12.343	12.387

cpu_samples_per_sec

n cellSizeMeters	120.0	500.0
10000	104.979	104.208
100000	112.562	112.750
300000	117.800	119.360

alloc_events_per_sec

n cellSizeMeters	120.0	500.0
10000	7156.917	7141.625
100000	3672.958	4319.750
300000	1947.000	1833.760

alloc_events_total

n cellSizeMeters	120.0	500.0
10000	171766	171399
100000	88151	103674
300000	48675	45844

alloc_tlab_share_pct

n cellSizeMeters	120.0	500.0
10000	100.000	100.000
100000	100.000	100.000
300000	100.000	100.000

Notes

- Фактически выполняется обычный Map.put по существующему ключу без обслуживания пространственной структуры.

Вывод

Пространственный индекс GeoSpatialIndex демонстрирует ожидаемый компромисс между затратами на поддержку структуры данных и эффективностью пространственного поиска. По сравнению с наивной реализацией он заметно дороже на этапах построения и изменения данных, поскольку каждое обновление требует не только работы с HashMap, но и сопровождения структуры QuadTree. Однако при выполнении запросов на поиск ближайших объектов индекс обеспечивает существенное ускорение, особенно при малых и средних радиусах поиска, так как вместо полного перебора всей коллекции обрабатывает только объекты-кандидаты из ограниченной пространственной области. При этом путь выполнения запроса в пространственном индексе сопровождается заметной активностью по созданию временных объектов из-за формирования промежуточных списков кандидатов, множества seenIds и объектов результатов. Следовательно, текущая реализация хорошо подходит для сценариев, в которых основную часть нагрузки составляют операции чтения и поиска, однако для сценариев с большим числом обновлений и с высокими требованиями к экономному использованию памяти могут потребоваться дополнительные оптимизации.